Sky: CapAutomator

Security Review

Cantina Managed review by:
Christoph Michel, Lead Security Researcher
M4rio.eth, Lead Security Researcher

February 18, 2026

Contents

1 Introduction **2**
1.1 About Cantina 2
1.2 Disclaimer 2
1.3 Risk assessment 2
1.3.1 Severity Classification 2

2 Security Review Summary **3**

3 Findings **4**
3.1 Informational 4
3.1.1 Large depositor can temporarily block deposits by lowering cap before `exec()` . . . 4
3.1.2 Use reserve-configuration decimals to mirror Aave cap math exactly 4

DRAFT

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

DRAFT

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Feb 2nd to Feb 4th the Cantina team conducted a review of sparklend-cap-automator on commit hash 3997e8bd (tag v1.1.0-beta.0). The team identified a total of **2** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	2	1	1
Total	**2**	**1**	**1**

The Cantina Managed team reviewed Sky's sparklend-cap-automator holistically on commit hash f115d10e (tag v1.1.0) and concluded that all findings were addressed and no new vulnerabilities were identified.

DRAFT

3 Findings

3.1 Informational

3.1.1 Large depositor can temporarily block deposits by lowering cap before `exec()`

Severity: Informational

Context: CapAutomator.sol#L170-L178

Description: Cap updates are derived from a supply snapshot. The mechanism enforces:

1. At most one cap update per block (`lastUpdateBlock` guard).
2. At most one cap increase per `increaseCooldown` window (`lastIncreaseTime` check).

Because decreases are not cooldown-limited, an attacker with sufficient deposited liquidity can withdraw before `exec()` to make the snapshot artificially low, force a lower cap, then re-deposit after `exec()` in the same block. This can leave minimal or zero headroom and cause user deposits to revert.

Recommendation: No code change recommended. Treat as an accepted risk and document it explicitly. Exploiting this requires a considerable pre-existing deposit and coordination with `exec()`.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Use reserve-configuration decimals to mirror Aave cap math exactly

Severity: Informational

Context: CapAutomator.sol#L189-L193, CapAutomator.sol#L225-L227, Aave reference (supply): ValidationLogic.sol#L80-L86, Aave reference (borrow): ValidationLogic.sol#L175-L193

Description: `CapAutomator` currently normalizes with token `decimals()` from `aToken` / `variableDebtToken`. This should be equivalent in practice, but differs from Aave's canonical implementation, which uses `reserveData.configuration.getDecimals()` directly.

Using the exact Aave expression improves auditability/maintainability (1:1 reasoning against upstream logic), avoids relying on implicit token-decimals parity assumptions, and is marginally more gas-efficient in this call path.

Recommendation: Consider replacing both denominators with

```
10 ** reserveData.configuration.getDecimals()
```

to exactly mirror Aave's cap-check math.

Sky: Fixed in commit d35834cb.

Cantina Managed: Fix verified.

DRAFT